Unit 3: Software Design

This PPT explains **how we plan software before writing code**. Think of it like building a house: before construction, we create a drawing that shows rooms, doors, wiring, and materials. In software, design is that drawing or **blueprint.** [1]

We will cover these topics one by one:

- Design concepts and principles
- Component-level design
- Function-oriented and object-oriented design
- Cohesion and coupling
- Software architecture
- User-interface design
- Web application design

Design basics

Software design starts after understanding requirements—what users and stakeholders want. It changes those requirements into a plan for coding, testing, and building the software. [1]

Design is important because without it, software can become difficult to change, test, understand, and maintain. A small change could unexpectedly break other parts of the system. [1]

A good design specifies:

- **Modules**: Separate parts of a system, such as Login, Payment, or Report.
- **Control relationship**: Which module calls or controls another module.
- **Interfaces**: What data modules exchange with each other.
- **Algorithms**: The step-by-step logic used by a module.
- **Data structures**: How data is stored and organized. [1]

The main design outputs are:

- **Data/Class design**: Classes and data structures.
- **Architectural design**: Major components and their connections.
- **Interface design**: How users and external systems communicate with software.
- **Component-level design**: Detailed internal logic of each module. [1]

Design principles

Design concepts are the basic ideas used to organize software well. Important concepts include abstraction, modularity, encapsulation, inheritance, polymorphism, cohesion, coupling, and design patterns.[1]

- **Abstraction**: Show only important details and hide unnecessary complexity. For example, an ATM user presses “Withdraw” without knowing how the bank server processes the request.
- **Modularity**: Divide a large system into smaller manageable modules.
- **Encapsulation**: Keep an object’s data and related functions together, while preventing direct unwanted access.
- **Inheritance**: A new class can reuse features from an existing class. For example, `Car` can inherit common properties from `Vehicle`.
- **Polymorphism**: The same action can behave differently for different objects. For example, a `draw()` method works differently for a circle and a rectangle.[1]

A good design should meet stated requirements, meet reasonable stakeholder expectations, be understandable to developers and testers, and show data, functions, and behavior clearly.[1]

Software quality

The PPT uses **FURPS** to remember key quality attributes:[1]

- **Functionality**: Does the software provide the required features correctly and securely?
- **Usability**: Is it easy, clear, attractive, and comfortable for people to use?
- **Reliability**: Does it work correctly over time and recover properly after failure?
- **Performance**: Does it respond quickly and use memory, CPU, and other resources efficiently?
- **Supportability**: Is it easy to update, adapt, repair, and extend later?

The goal is software with **firmness**, **commodity**, and **delight**: it should work without bugs, be useful for its purpose, and feel pleasant to use.[1]

Component-level design

A **component** is a modular software building block, such as a class, function, module, or service. Component-level design describes its internal logic, its inputs and outputs, and how it interacts with other components.[1]

For example, in an online shopping app, a `PaymentService` component may:

- Receive order and payment details.
- Validate the payment.
- Communicate with a payment gateway.
- Return success or failure.

A well-designed component can be developed, tested, replaced, and reused more independently. [1]

Two design approaches

Function-oriented design organizes software around functions or procedures. It breaks work into steps such as "take input," "process data," and "produce output." A **Data Flow Diagram (DFD)** is commonly used to show how data moves between these functions. This approach fits systems with a clear input-process-output flow. [1]

Example: In a result-calculation system:

- Read student marks.
- Calculate total.
- Calculate percentage.
- Display grade.

Object-oriented design (OOD) organizes software around real-world objects. Every object contains **data** (attributes) and **actions** (methods). It uses classes and their interactions, often shown using UML class diagrams and sequence diagrams. [1]

Example: In a library system, `Book`, `Member`, and `Librarian` can be objects. A `Book` may have title and author as data, plus methods such as `issueBook()` or `returnBook()`.

Coupling

Coupling means how strongly one module depends on another module. The aim is **low coupling**—modules should depend on each other as little as possible. [1]

- **High coupling**: One module directly knows too much about another module. A change in one can break the other.
- **Low coupling**: Modules communicate through clear, limited interfaces, so one can change without heavily affecting the rest.

For example, if a screen directly accesses database details, changing the database can break the screen. A better design puts a **Repository** between them: the screen talks to the repository, and the repository handles the database or cloud API. [1]

Two types in the PPT:

- **Data coupling**: Pass only required data, such as `email` and `password`. This is good.
- **Stamp coupling**: Pass a whole object when only one small value is needed, such as sending the complete `User` object only to get its email. This creates unnecessary dependency. [1]

Low coupling helps independent testing, reuse, teamwork, maintainability, scalability, and safer changes. [1]

Cohesion

Cohesion means how closely the tasks inside one module are related. The aim is **high cohesion**—one module should focus on one clear purpose.[1]

- **High cohesion:** A `LoginViewModel` contains functions only related to login, such as validating input, logging in, and handling the login result.
- **Low cohesion:** An `AppUtilityClass` handles login, date formatting, API calls, and theme settings together. These jobs are unrelated, so the class becomes confusing and difficult to reuse.[1]

Remember this simple exam line: **Good software design has high cohesion and low coupling.** High cohesion means "one module, one focused job"; low coupling means "modules are not overly dependent."[1]

Software architecture

Software architecture is the high-level structure of the entire system: its main components, their visible properties, and the relationships between them.[1]

Architecture helps stakeholders communicate, lets developers compare design choices early, checks whether requirements are met, and reduces project risk before much coding is done.[1]

The PPT shows a **three-tier architecture**:

- **Presentation layer (UI layer):** What the user sees and uses, such as web pages or mobile screens.
- **Business logic layer (service layer):** The core rules, calculations, and decisions of the application.
- **Data access layer:** Connects to databases and handles storing or retrieving data.[1]

For an online store:

- UI layer: Product page and checkout screen.
- Business layer: Calculates discounts, tax, and order total.
- Data layer: Saves orders and reads product information from the database.

User-interface design

User Interface (UI) design creates the communication method between humans and computers. It decides screens, buttons, forms, menus, messages, and layouts.[1]

The PPT gives three golden rules:

1. **Place the user in control**
   Users should be able to make choices, cancel tasks, undo mistakes, and navigate freely. For example, an app should provide an **Undo** option after deleting something. Advanced users may use shortcuts, while new users may use buttons and menus.[1]

2. **Reduce the user's memory load**
   Do not force users to remember too much between screens. Use clear labels, familiar icons, visible instructions, sensible default values, and easy shortcuts such as `Ctrl + P` for print. Show basic information first and additional details only when needed; this is called **progressive disclosure.** [1]
3. **Make the interface consistent**
   Similar buttons, colors, fonts, labels, and actions should behave the same throughout the application. For example, users expect `Ctrl + S` to save, so changing that shortcut without a strong reason creates confusion. [1]

Web application design

A **WebApp** is an application used through a web browser, such as an e-commerce website, online banking site, or college portal. WebApp design includes both technical planning and visual/user-experience planning. [1]

The PPT's WebApp design pyramid includes:

- **Content design**: Organizing text, images, videos, and other content.
- **Aesthetic design**: Making the website visually attractive for its target users.
- **Interface design**: Creating clear screens, controls, and consistent interaction.
- **Navigation design**: Helping users move easily between pages and features.
- **Architecture design**: Organizing content and functions at the overall system level.
- **Component design**: Building the modules that process data, create dynamic content, and connect to databases or external systems. [1]

For example, on an online shopping website, navigation helps users move from Home to Categories to Product to Cart; content design organizes product information; and component design processes payments and updates stock. [1]

Start by explaining, in your own words: why do you think a software project needs a design before developers begin coding?

⁂

1. unit-3.pptx